Southern New Hampshire University

"

CS 499 Milestone Three Narrative

Giuseppe Claudio Zema

07/21/2026

The artifact selected for this enhancement is the Grazioso Salvare animal rescue management system originally developed for my CS 340 Client/Server Development project during C6-2025 (November/December) at SNHU. The original application was a JupyterDash dashboard connected to a MongoDB database containing animal shelter records. Users could view animal information, filter records, and visualize data through dashboard components.

During Milestone Two, the artifact was transformed into a MEAN-stack application consisting of an Angular frontend, Express.js REST API, and MongoDB database. That enhancement focused primarily on software engineering practices, architecture, security, authentication, and usability improvements. Milestone Three continued the evolution of the artifact by focusing specifically on algorithms and data structures to improve application performance, scalability, and efficiency when working with larger datasets.

I selected this artifact because it represents the most complete example of my growth throughout the Computer Science program. The project evolved from a simple academic dashboard into a full-stack application that incorporates frontend development, backend services, database management, security practices, testing, and now algorithmic optimization.

The original dashboard was designed around a relatively small dataset and performed some data processing through client-side operations. While this approach worked for the initial project requirements, it created scalability limitations because unnecessary data could be transferred and

processed outside the database layer. The purpose of this enhancement was to evaluate the existing design and apply appropriate algorithms and data structures to improve efficiency.

The first enhancement was the implementation of a MongoDB compound index using the fields animal_type, breed, sex_upon_outcome, and age_upon_outcome_in_weeks. These fields represent common filtering criteria used by the dashboard. The compound index allows MongoDB to use indexed lookups rather than performing unnecessary collection scans. Query execution was evaluated using MongoDB's explain("executionStats") functionality to compare database behavior before and after index implementation.

The second enhancement focused on moving data processing responsibilities closer to the database layer. MongoDB aggregation pipelines were implemented to replace unnecessary client-side processing. The breed distribution functionality now uses aggregation stages including $match, $group, $sort, and $project to filter records, calculate grouped results, sort output, and return only the required fields. This reduces network overhead and allows the database engine to perform operations more efficiently.

The third enhancement introduced server-side pagination using MongoDB's skip() and limit() operations. Previously, transferring larger datasets and processing records on the client side could become inefficient as the database grew. Server-side pagination allows the application to request only the records required for the current view, reducing memory consumption and improving responsiveness. While skip/limit pagination provides a simple and effective solution, I also evaluated the trade-off with cursor-based pagination, which can provide better performance for very large datasets but requires additional implementation complexity.

The final enhancement implemented a custom Least Recently Used (LRU) cache using JavaScript's Map data structure. The cache stores recently requested query results so repeated dashboard requests can avoid unnecessary database operations. JavaScript Map was selected because it maintains insertion order while providing efficient lookup, insertion, and deletion operations. This allowed the cache to remove the least recently used entry without requiring an additional tracking structure. Cache invalidation was added to create, update, and delete operations to ensure that stale data is not returned after database modifications.

This enhancement primarily supports Course Outcome Three:

Design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to its solution, while managing the trade-offs involved in design choices.

The enhancement demonstrates this outcome by selecting and implementing multiple algorithmic improvements. The compound index demonstrates understanding of efficient data retrieval structures. The aggregation pipeline demonstrates selecting the appropriate processing location between the client and database layers. The pagination implementation demonstrates evaluating scalability considerations. The LRU cache demonstrates selecting a data structure based on operational requirements rather than convenience. These enhancements were also considered from a security standpoint: pagination parameters are constrained to prevent excessive or malformed requests, and cache invalidation on write operations ensures that outdated or sensitive records are not inadvertently served from stale entries.

The enhancement also supports Course Outcome Four by applying professional software development practices, including database optimization, automated testing, maintainable backend architecture, and performance evaluation.

No changes are necessary to the original outcome-coverage plan established in Module One. Milestone Two addressed the architectural and security portions of the project, while this enhancement completed the planned focus on algorithms and data structures.

Completing this enhancement changed my understanding of algorithms and data structures from primarily theoretical concepts into practical software engineering tools. Concepts such as indexing, caching, and algorithmic complexity became tangible as I implemented them to improve application performance and scalability. One of the main challenges was deciding where processing should occur. The original dashboard performed much of its work on the client, but moving filtering, grouping, sorting, and pagination into MongoDB demonstrated how server-side processing reduces unnecessary data transfer and improves efficiency.
Implementing the LRU cache introduced another important consideration. While caching improves performance by reducing database queries, it also requires proper cache invalidation to prevent stale data from being returned. This reinforced the importance of balancing correctness with performance. Overall, this enhancement strengthened my ability to evaluate trade-offs involving execution speed, memory usage, implementation complexity, and maintainability. It demonstrated that effective software development depends on selecting algorithms and data structures that best fit the problem rather than simply implementing the first available solution.